A Graph-based Framework for Coverage Analysis in Autonomous Driving

Thomas Mühlenstädt[1][0009−0002−9978−0504] and Marius Bause[1][0000−0003−4105−8906]

Independent Researcher
tmuehlenstaedt@gmail.com mariusbause@gmx.de

Abstract. Coverage analysis is essential for validating the safety of autonomous driving systems, yet existing approaches typically assess coverage factors individually, struggling to capture the complex interactions inherent in traffic scenes. This paper proposes a graph-based framework that represents traffic scenes as hierarchical graphs combining maps with actor relationships, using a two-phase construction algorithm to capture spatial configurations such as leading, following, neighboring, and opposing. We present two complementary coverage methods: a subgraph isomorphism approach that matches scenes against manually defined archetype graphs, and a graph embedding approach using Graph Isomorphism Networks with Edge features (GINE) trained via self-supervised contrastive learning. Validated on real-world Argoverse 2.0 data and synthetic CARLA data, the isomorphism method yields node coverage percentages from predefined archetypes, while the embedding approach reveals latent structure suitable for clustering and anomaly detection. The framework scales efficiently across diverse scenarios without scenario-specific handling and naturally accommodates varying numbers of actors.

Keywords: Coverage Analysis · Autonomous Driving · Traffic Scene Graphs · Subgraph Isomorphism · Graph Neural Networks

1 Introduction

In autonomous driving, coverage analysis is a crucial step to ensure the safety and reliability of the system. At its core, a coverage argument asks whether the scenes observed so far have exhausted the space of traffic scenes that can occur in the real world. In most situations, such arguments are collected either per coverage factor, or maybe up to 2 or 3 factor interactions. See for example [8] for a production-grade implementation of state-of-the-art coverage analysis.

In contrast to existing approaches, this paper proposes a graph-based framework for coverage analysis. The framework takes the *complete traffic scene* as the unit of coverage rather than an individual factor: each scene is represented as a single graph whose nodes carry the parameters describing the individual actors and whose edges carry the parameters describing the relations between them.

The key advantage of this representation is that arbitrarily complex scenes—with varying numbers of actors and the higher-order interactions between them—are captured naturally within one object, instead of being decomposed into isolated coverage factors or low-order factor combinations. Coverage is therefore assessed against whole scenes: we ask how completely the observed scene graphs cover those that can occur in reality. While graph-based representations of traffic scenes already exist, they have not been specifically designed for this coverage analysis. This work bridges that gap by utilizing graph-based traffic scene representations explicitly for coverage analysis.

This paper is concerned with completeness of the observed scenes, not with the safety of the system. A reasonable strategy is to focus coverage analysis on the more safety-critical scenes, but here the focus is solely on coverage, and we leave safety considerations to future work.

2 Existing coverage and analysis approaches

Foundational terminology for automated driving is established by the SAE J3016 taxonomy [18], which defines six levels of driving automation and key concepts such as the Dynamic Driving Task (DDT) and Operational Design Domain (ODD). Building on this, [23] provide precise definitions for the commonly conflated terms *scene*, *situation*, and *scenario*, enabling a consistent vocabulary for test design and coverage arguments. Complementing this work, [9] propose an object-oriented framework for scenario definition that ensures the scenarios constructed are both complete and comparable.

Scenario-based testing has emerged as the dominant paradigm for validating automated driving functions. The PEGASUS project [19] introduces a six-layer model to decompose the driving environment into structured logical scenarios, shifting validation from public road testing toward systematic scenario exploration across simulation and field tests. [13] refine this with three abstraction levels—functional, logical, and concrete scenarios—while noting that existing parameter-selection methods lack a systematic way to determine meaningful test coverage. Motivated by this weak point, [20] present trajectory-based clustering of real-world driving data to structure the scenario space and reduce test-set redundancy. Complementing such data-driven structuring, [27] generate adversarial test scenes that explicitly target harmful, rare, and ambiguous situations for autonomous-driving testing.

A central challenge is the *approval trap* identified by [25]: statistically demonstrating superior safety through real-world driving alone would require billions of test kilometres, motivating simulation-based tools and systematic coverage methods. [6] tackle this bottleneck directly with a dense reinforcement-learning environment that accelerates safety validation by orders of magnitude. General testing concepts [1] are adapted for the domain in [8] through coverage buckets and performance metrics, while ISO 21448 [22] and UL 4600 [11] standardize coverage criteria for automated driving.

More recent work represents traffic scenes directly as graphs to enable scalable, label-free analysis of the scenario space. [28] encode semantic scene graphs into low-dimensional embeddings via a Siamese network to cluster traffic scenes without manual labels, an approach refined through graph-based contrastive learning in [29] and extended to heterogeneous scenario graphs for clustering and similarity search by [17]. Targeting scalability, [15] propose SparScene, a sparse graph-learning framework that exploits lane-graph topology rather than distance thresholds to build structure-aware actor–map connections, scaling scene representation to thousands of actors for large-scale trajectory generation, with related work applying graph structure learning to traffic-scene representation and encoding more broadly [16]. Cluster-based analyses have likewise been used to assess the completeness of scenario categories [21], complementing the standardized coverage criteria discussed above. These graph-based representations directly motivate our framework, which combines explicit subgraph isomorphism against archetype graphs with learned GINE embeddings to quantify coverage of the scenario space.

3 Defining a traffic scene graph

3.1 Datasets

Argoverse 2.0 [26] is a large-scale dataset of 250,000 scenarios (11 s each) with tracked trajectories for vehicles, pedestrians, and cyclists, plus semantic map information (lane boundaries, crosswalks, traffic signals) across diverse geographic locations. A subset of 17,000 training scenes is used.

CARLA (Car Learning to Act, [4]) is an open-source simulator for autonomous driving research. Version 0.9.15 is used across six maps (Town01–Town05, Town07) with 20–60 vehicles per run, simulating mixed traffic with varied vehicle types and behavioural parameters. The resulting data consists of 2,050 scenes of 11 seconds each; actors spread across the map may form multiple disconnected components per scene.

3.2 Map Graph Construction

Following established practice of representing the road network as an explicit lane graph [12], the map graph is a directed multigraph $G_{\text{map}} = (V_{\text{map}}, E_{\text{map}})$ where each node $v \in V_{\text{map}}$ represents a lane segment, storing geometric information (boundaries, centerline, length) and semantic attributes (intersection status, road and lane type). Edges encode three spatial relationships between lanes: **following edges** connect lanes forming a continuous path in the same direction, **neighbor edges** connect adjacent lanes traveling in the same direction, and **opposite edges** connect lanes traveling in opposite directions. The map graph is constructed by processing map data from Argoverse or CARLA to extract these relationships. Lanes with geometric overlap are marked as intersection lanes.

3.3 Actor Graph Construction

The actor graph $G_{\text{actor}} = (V_{\text{actor}}, E_{\text{actor}})$ represents dynamic relationships between actors at a specific timestep. Like LaneGCN's coupling of actor–map and actor–actor interactions over the lane graph [12], these actor relationships are derived from the underlying lane network. Each node $v \in V_{\text{actor}}$ represents an actor and stores attributes including primary lane ID, all occupied lane IDs, longitudinal position s along the lane centerline, 3D position, longitudinal speed, actor type, and a lane change indicator. Each edge $e \in E_{\text{actor}}$ stores a actor-actor relation type and a path length along the lane network. Although each actor graph is built at a single timestep, the lane change indicator already encodes a one-step temporal signal: it is set by comparing an actor's lane to its lane one timestep earlier ($\Delta t = 1\,\text{s}$), so dynamic maneuvers such as cut-in and cut-out are captured even within a single-timestep graph.

Four relationship types are defined between actors, ordered by semantic hierarchy:

1. **Following/Leading**: Longitudinal relationships where actors share the same lane or are connected entirely by following edges.
2. **Neighbor**: Lateral relationships via paths containing exactly one neighbor edge (actors need not be on immediately adjacent lanes).
3. **Opposite**: Relationships via paths containing exactly one opposite edge (actors need not be on immediately opposite lanes).

The attributes and relation types defined above are one example instantiation of the graph model. The parameters of interest, the actor types, and the actor relations are free to be chosen by the user. The coverage methodology is unaffected by these modelling choices.

3.4 Hierarchical Graph Construction Algorithm

The actor graph is constructed in two phases that separate relation discovery from graph building, enabling hierarchical processing and preventing redundant edges. All parameters are listed in Table 1.

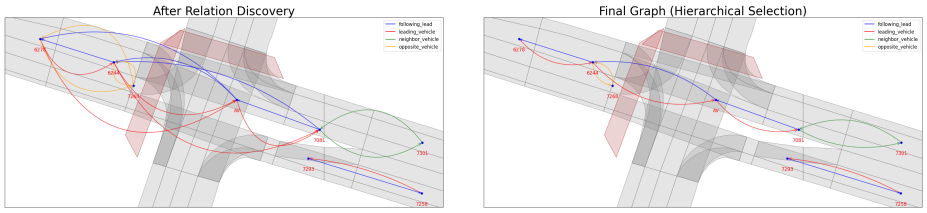
Table 1: Input parameters for actor graph construction, per relation type. Max. distance is the Euclidean limit used in the discovery phase (forward/backward); max. path length bounds the number of map-graph edges in a connecting path during construction. The timestep increment is $\Delta t = 1.0\,\text{s}$.

Relation	**Max. distance fwd/bwd [m]**	**Max. path length [edges]**
Leading/following	100	3
Neighbor	50 / 50	2
Opposite	100 / 10	2

Phase 1: Relation Discovery: For each actor pair (A, B) at timestep t, the algorithm determines their primary lanes and checks if a connecting path exists in the map graph. The path structure determines the relationship type: paths consisting entirely of following edges yield following/leading relations, paths with exactly one neighbor (or opposite) edge yield neighbor (or opposite) relations. Both lane-based path length and Euclidean distance are checked against the thresholds in Table 1 to filter distant actors.

Phase 2: Hierarchical Graph Construction: Edges are added in hierarchical order—leading/following first (highest priority), then neighbor, then opposite—each sorted by path length (shortest first). Before adding an edge between actors A and B, a breadth-first search checks whether a path of length $\leq$ max_node_distance already exists in the current graph. If so, the direct edge is skipped, as the relationship is already implicitly encoded. The graph is updated after each edge addition so that subsequent checks reflect the current state.

This redundancy prevention significantly reduces edges while preserving connectivity. For example, three vehicles in a row need only two lead-follow edges; the third pairwise relation is encoded through the intermediate vehicle. Similarly, for a row of vehicles adjacent to an opposite-direction vehicle, only the closest pairwise relation needs an explicit edge (see Figure 1). This is especially beneficial in dense traffic, where pairwise relations grow quadratically with the number of actors: pruning keeps the graph and the isomorphism checks tractable while the multi-actor interaction remains fully represented. Where every pairwise relation must stay explicit, the pruning can be gated by a lower bound on the implicit path length (a local-density threshold), so the closest, most safety-relevant relations are always retained.



(a) After relation discovery (b) After hierarchical selection

Fig. 1: The discovery graph (left) contains all relations within distance limits. Hierarchical selection (right) removes redundant edges representable through existing paths.

4 Create subgraphs for coverage analysis

Traffic scene archetypes from the literature (e.g., lead vehicle following, opposite traffic) can be expressed as small graphs. To determine whether such an archetype appears in a larger traffic scene graph G, we check if any subgraph of G is isomorphic to the archetype graph A. While subgraph isomorphism is NP-hard in general, the graphs considered here are small enough for practical computation using the VF2 algorithm [3].

Our strategy is as follows: (1) Define a set of archetype subgraphs S representing relevant traffic situations. (2) Specify which node and edge attributes are considered for the isomorphism check. (3) For each traffic scene graph G (e.g., from CARLA or Argoverse), check if any subgraph of G is isomorphic to any archetype in S and record the result in a coverage table C.

This bottom-up approach starts from individual situation archetypes and checks their presence across datasets. The resulting coverage table enables follow-up analyses such as visualizing attribute distributions (speed, distance) for matched scenes, cross-tabulating co-occurring archetypes, or computing AV performance metrics conditioned on specific archetypes.

We defined 18 subgraph archetypes covering common traffic scenarios: simple 2-actor patterns (following, opposite, neighbor, used only for isolated pairs), 3-actor patterns (lead vehicle with neighbor, platoons, opposite traffic, lane change/cut-in scenarios with intersection variants), 4-actor patterns (cut-out, multi-vehicle platoons, lead-follow with opposite traffic and intersection variants), and 5-actor patterns combining lead, neighbor, and opposite vehicles with intersection variants. These archetypes are a *user-defined input*, drawn from common scenario primitives in the literature (lead–follow, cut-in, cut-out, opposite traffic, platoons) and the authors' experience; they illustrate the method rather than form a complete or safety-certified taxonomy, and practitioners can substitute archetypes specific to their operational design domain. A real safety case would typically require dozens or more, and assuring completeness of the set is then a critical concern.

In a real safety case, the archetypes should be derived top-down from established processes rather than expert intuition: a Hazard Analysis and Risk Assessment (HARA, ISO 26262) yields the hazardous traffic situations to be covered, while UL 4600 requires an explicit argument that the scenario space is sufficiently covered. Each identified hazard maps to an archetype, and any hazard without a matching archetype directly flags a coverage gap and thus a risk of missed hazards.

The complementary embedding approach (Section 5) mitigates this by operating *without* predefined archetypes, surfacing scenes that match no archetype. The last step in the analysis of the subgraphs is to identify coverage differences between real-world and simulation datasets. The archetypes provide an intuitive understanding of what traffic situations are missing from the simulated environment.

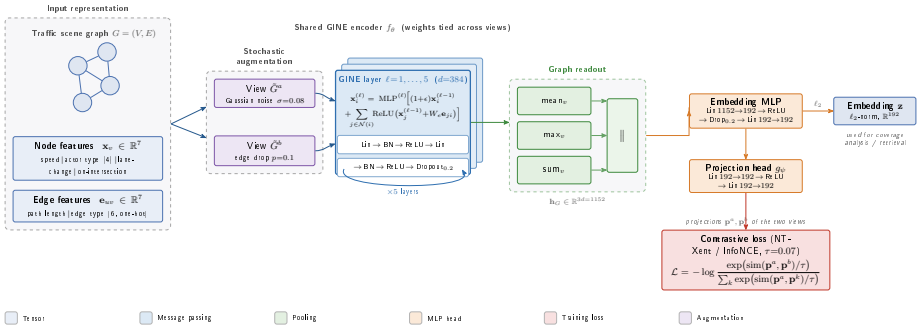


Fig. 2: Architecture of the GINE-based graph embedding model and its self-supervised training. A traffic scene graph G carries 7-dimensional node features (speed, actor type, lane-change and intersection flags) and 7-dimensional edge features (path length and one-hot edge type). Two stochastically augmented views (Gaussian feature noise $\sigma = 0.08$, edge dropout $p = 0.1$) are passed through a weight-shared encoder f_θ of five GINE layers ($d = 384$), each performing edge-conditioned message passing followed by batch normalisation, ReLU and dropout. Node states are aggregated by concatenated mean/max/sum pooling ($\mathbb{R}^{1152}$), mapped by an ℓ_2-normalised embedding MLP to the scene embedding $\mathbf{z} \in \mathbb{R}^{192}$ used downstream, and by a projection head g_ψ to the space where the NT-Xent/InfoNCE contrastive loss ($\tau = 0.07$) is computed between the two views.

5 Graph Embeddings for Traffic Scene Analysis

The subgraph isomorphism approach from Section 4 provides a bottom-up methodology for analyzing traffic scenarios through predefined archetypes. However, real-world traffic scene graphs exhibit significantly higher complexity than these patterns, motivating complementary top-down approaches such as graph embeddings.

Embeddings translate raw data into a vector space where distances measure similarity, analogous to the Word2Vec model for words [14]. For traffic scene graphs, embeddings enable coverage analysis by comparing scenes: identifying similar simulation–real-world scenario pairs, detecting near duplicates, or visualizing structures that would be intractable in the original space of all possible traffic scenes.

A Graph Isomorphism Network with Edge features (GINE) [10] is used to generate embeddings, implemented in PyTorch Geometric [7]. GINE is chosen because it allows embedding of edge features, so we may encode actor relation type and distance as integral descriptors for a traffic scene (see Figure 2); attention-based architectures such as graph attention networks [24] are a common alternative but do not natively encode edge attributes in their original formulation.

The node and edge features and the full architecture are summarised in Figure 2. The model was trained on CARLA and Argoverse 2.0 using self-supervised contrastive learning [2,10]: for each mini-batch two augmented views are created, and the contrastive loss maximises their agreement while contrasting against the other in-batch graphs. Optimization used AdamW (weight decay 5×10^{-6}), learning-rate warmup over three epochs, and exponential decay (initial rate 0.0015, factor 0.85) across 15 training stages [7].

6 Application: Analysing traffic scenes

We apply both methods to CARLA and Argoverse data to identify coverage gaps, targeting not the exact real-world distribution but gaps that could lead to insufficient testing of safety-critical scenarios.

6.1 Subgraph Isomorphism Coverage Analysis

We match the 18 archetypes (Table 2) against each scene graph and compare CARLA and Argoverse across two gap dimensions: structural presence of scenario types and parameter distributions within matched scenarios.

Table 2: Overview of all subgraph archetypes with their structural properties. Edge types: *fl* = following_lead, *lv* = leading_vehicle, *nv* = neighbor_vehicle, *ov* = opposite_vehicle.

Group	Archetype	Actors	Edges	Edge Types
Simple (2-actor)	Simple Following	2	2	fl, lv
	Simple Opposite	2	2	ov
	Simple Neighbor	2	2	nv
Complex (3-actor)	Lead + Neighbor (intersection)	3	4	fl, lv, nv
	Lead + Neighbor	3	4	fl, lv, nv
	Cut-in	3	4	fl, lv
	Cut-in (intersection)	3	4	fl, lv
	Platoon (intersection)	3	4	fl, lv
	Opposite Traffic (intersection)	3	4	fl, lv, ov
	Lead + Neighbor at Intersection	3	4	fl, lv, nv
	Triple Opposite (intersection)	3	4	ov
	Lead + Following in Back	3	4	fl, lv
Complex (4-actor)	Cut-out	4	6	fl, lv, nv
	Cut-out (intersection)	4	6	fl, lv, nv
	4-Vehicle Platoon (intersection)	4	6	fl, lv
	4-Vehicle Opposite (intersection)	4	6	fl, lv, ov
Complex (5-actor)	Lead + Neighbor + Opposite	5	8	fl, lv, nv, ov
	Lead + Neighbor + Opposite (inters.)	5	8	fl, lv, nv, ov

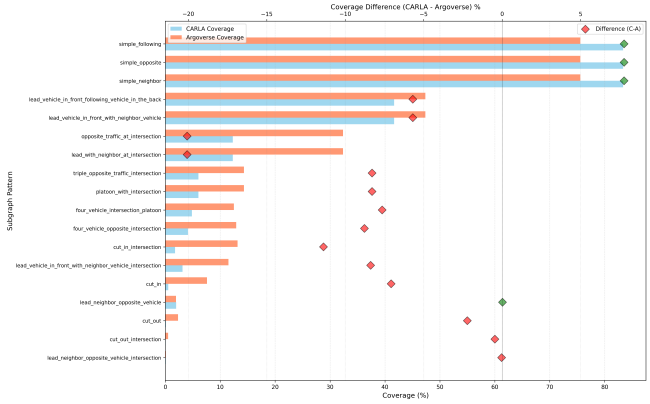


Fig. 3: Scenario archetype coverage comparison between CARLA and Argoverse datasets. Horizontal bars show absolute coverage percentages, while diamond markers indicate coverage differences (CARLA minus Argoverse).

Individual Scenario Archetype Coverage Gaps: Figure 3 presents a comprehensive comparison of scenario archetype coverage between CARLA and Argoverse datasets. The dual-axis visualization displays coverage percentages (horizontal bars) alongside coverage differences (diamond markers). Red diamonds indicate scenarios that are more prevalent in Argoverse, while green diamonds mark scenarios with higher CARLA representation.

The coverage distribution reveals a clear pattern: CARLA achieves higher coverage only for the three simple two-actor scenarios (`simple_following`, `simple_neighbor`, `simple_opposite`), as indicated by green diamond markers. For all complex multi-actor scenarios, Argoverse exhibits substantially higher occurrence rates (red diamond markers), demonstrating that the simulation systematically fails to generate complex traffic situations at the same frequency as observed in real-world data.

The identified coverage gaps in CARLA are substantial. The most critical gaps appear in intersection-related archetypes: `cut_out_intersection` shows nearly complete absence in CARLA (less than 1% coverage) while appearing in approximately 8% of Argoverse scenes. Similarly, `lead_neighbor_opposite_vehicle_intersection` appears in roughly 6% of Argoverse data but is virtually absent in CARLA. The `cut_in_intersection` pattern exhibits a comparable gap of approximately 7 percentage points. Both `cut_in` and `cut_out` scenarios show significant underrepresentation in CARLA regardless of intersections, indicating that the traffic generation script lacks the dynamic lateral maneuvers characteristic of real-world driving.

Parameter Distribution Gaps: Even when a scenario archetype is structurally present in both datasets, continuous parameter distributions can reveal additional coverage gaps. Figure 4 shows role-specific speed distributions within the `lead_vehicle_in_front_with_neighbor_vehicle` scenario.

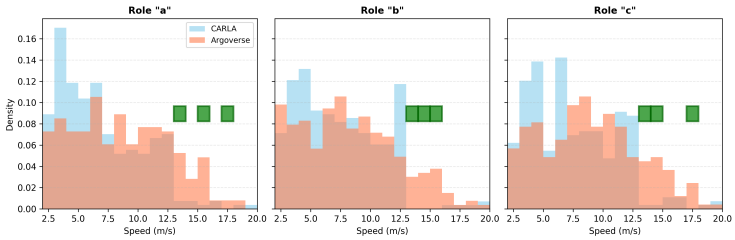


Fig. 4: Role-specific speed distribution comparison for the lead vehicle in front with neighbor vehicle scenario. Only actors with speed $\geq 2\,\mathrm{m/s}$ are considered, binned into fixed $1\,\mathrm{m/s}$ intervals, and the histograms are density-normalised. Green rectangles mark identified coverage gaps, defined per bin as a normalised Argoverse density ≥ 0.005 together with a CARLA density below 15% of it. I.e. Argoverse shows sufficient density while CARLA is nearly empty. Role "a" = ego vehicle, role "b" = lead vehicle, role "c" = neighbor vehicle.

CARLA concentrates actors at lower speeds (peak around 3–5 m/s) while Argoverse distributions extend to higher speeds (15–20 m/s and beyond). Coverage gaps consistently appear in the 13–17 m/s range for all three roles, demonstrating that coverage gaps exist both at the structural level (which scenarios are present) and at the parameter level within structurally matching scenarios.

The structured, bottom-up approach presented here demonstrates that subgraph isomorphism can effectively characterize traffic scene coverage for predefined archetypes. In a real application, these gaps could now e.g. be checked for safety critical aspects and prioritized accordingly, e.g. by analyzing the distribution of TTC values in the gaps. However, the manual definition of archetypes and the computational cost of isomorphism checking for large scenario collections motivate the graph embedding approach explored in the following paragraph.

6.2 Graph Embeddings Coverage Analysis

The GINE model (Section 5) is trained jointly on CARLA and Argoverse. As a plausibility check, we verified by visual inspection that the nearest neighbours of a query scene in embedding space consistently share its actor configuration and road geometry, indicating the model captures scene structure rather than superficial features.

The embedding space is analysed using PCA and t-SNE as shown in Figure 5. The first two principal components already explain 40% of the total variance. Both projections reveal that CARLA and Argoverse scenarios, while sharing a common region corresponding to simple following situations, differ substantially in their density distributions: For the Carla data, the main peak of the distribution is concentrated at higher values of the first principal component, while for Argoverse data the main peak is located at negative values of the first principal component. This indicates that the two datasets differ in their overall scenario

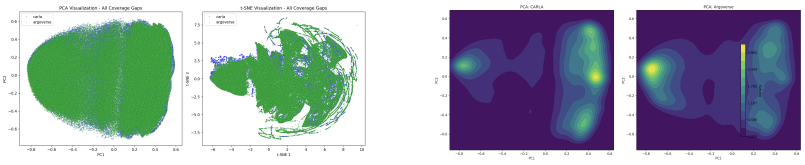


Fig. 5: Left: PCA and t-SNE visualisation of the embedding space for CARLA and Argoverse 2.0 scenarios. Right: PCA kernel density plots for Argoverse 2.0 scenarios, highlighting regions of high real-world density.

characteristics, with CARLA scenarios clustering in a different region of the embedding space than Argoverse scenarios.

Coverage Gap Identification in Embedding Space: No binning is required here, as the full graph is embedded. To quantify coverage gaps, Argoverse graphs whose subgraph isomorphism labels indicate under-represented scenario types are projected into the joint embedding space and compared against the CARLA density. 23,148 Argoverse graphs are identified as coverage gaps, dominated by `cut_in` (∼94% of all Argoverse cut-in instances), `cut_out`, `cut_out_intersection`, and `lead_neighbor_opposite_vehicle_intersection`. These gap graphs cluster in embedding regions where CARLA density is negligible (Fig. 6), reinforcing the gaps identified by subgraph isomorphism. This agreement between the embedding and the independent, exact subgraph-isomorphism labels (23,148 graphs, ∼94% of all cut-ins) constitutes a quantitative, label-grounded validation rather than a purely visual one; a formal retrieval-precision and clustering-quality benchmark against these archetype labels, together with embedding-based baselines, is the natural next step for which the ground-truth labels are already available.

The embedding is used as a coverage-screening layer, not a standalone safety oracle: every gap it flags is cross-checked against the interpretable subgraph labels, which act as an auditable ground truth. Trusting it for safety-critical decisions would additionally require calibrated out-of-distribution guarantees—e.g. conformal or SafeML-style distance-to-training-distribution bounds on the embedding—which we leave to future work.

7 Summary

This paper presented a graph-based framework for coverage analysis in autonomous driving that represents traffic scenes as hierarchical graphs combining map topology with actor relationships. Two complementary analysis approaches were developed: subgraph isomorphism using manually defined archetype graphs for interpretable pattern-based coverage metrics, and graph embeddings via GINE networks trained with contrastive learning for similarity-based assessment, clustering, and anomaly detection.

Both approaches were validated on Argoverse 2.0 and CARLA data. It was shown that hierarchical graphs capture complex traffic situations efficiently and

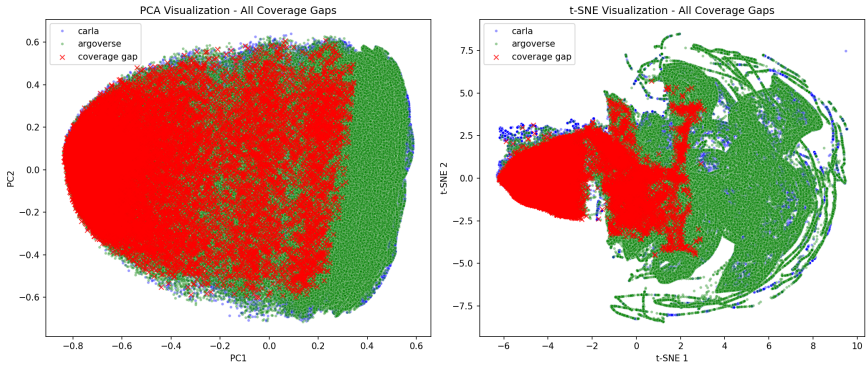


Fig. 6: Coverage gap visualisation in the embedding space. CARLA (blue) and Argoverse (green) scenarios are shown in PCA (left) and t-SNE (right) projections. Red crosses mark the 23,148 Argoverse graphs identified as coverage gaps, concentrated in regions where CARLA density is negligible.

allow the user to define individual specifics on definitions like follow vehicle, opposing vehicle and the set of relevant parameters.

We demonstrated that the graphs capture meaningful traffic scene structure and reveal differences between two datasets (Section 6). First, the bottom-up subgraph approach revealed differences in traffic scenes, combination of traffic scenes and distribution of scenario parameters. Second, the top-down graph embedding approach revealed differences in the embedding space, allowing for further analysis without manual definition of archetypes. This representation can be the foundation for further analysis like similarity search, clustering and anomaly detection. Compared to approaches like TNO Streetwise [5], the embedding approach scales efficiently without scenario-specific handling rules and naturally accommodates varying numbers of actors.

Future work will incorporate temporal information through multi-timestep graph structures, capturing the full dynamics of maneuvers such as cut-in, cut-out, braking, and yielding more faithfully than a single-timestep graph, and close the loop on archetype completeness by automatically extracting new archetype candidates from real-world scenes that populate dense embedding regions yet match no predefined archetype. The source code is publicly available at `https://github.com/tmuehlen80/graph_coverage`.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ammann, P., Offutt, J.: Introduction to Software Testing. Cambridge University Press (2008)

2. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: International Conference on Machine Learning (ICML). pp. 1597–1607. PMLR (2020)
3. Cordella, L.P., Foggia, P., Sansone, C., Vento, M.: A (sub)graph isomorphism algorithm for matching large graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence **26**(10), 1367–1372 (oct 2004)
4. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning. pp. 1–16 (2017)
5. Elrofai, H., Paardekooper, J.P., de Gelder, E., Kalisvaart, S., Op den Camp, O.: StreetWise: Scenario-based safety validation of connected and automated driving. Tech. rep., TNO, Helmond, The Netherlands (2018), https://publications.tno.nl/publication/34626550/AyT8Zc/TNO-2018-streetwise.pdf
6. Feng, S., Sun, H., Yan, X., Zhu, H., Zou, Z., Shen, S., Liu, H.X.: Dense reinforcement learning for safety validation of autonomous vehicles. Nature **615**(7953), 620–627 (2023). https://doi.org/10.1038/s41586-023-05732-2
7. Fey, M., Lenssen, J.E.: Fast graph representation learning with PyTorch Geometric. In: ICLR Workshop on Representation Learning on Graphs and Manifolds (2019)
8. Foretellix: Av coverage and performance metrics. https://blog.foretellix.com/2019/09/13/av-coverage-and-performance-metrics/ (September 2019), accessed: 2025-04-04
9. de Gelder, E., Paardekooper, J.P., Saberi, A.K., Elrofai, H., Camp, O.O.d., Kraines, S., Ploeg, J., De Schutter, B.: Towards an ontology for scenario definition for the assessment of automated vehicles: An object-oriented framework. IEEE Transactions on Intelligent Vehicles **7**(2), 300–314 (2022). https://doi.org/10.1109/TIV.2022.3144803
10. Hu, W., Liu, B., Gomes, J., Zitnik, M., Liang, P., Pande, V., Leskovec, J.: Strategies for pre-training graph neural networks (2020), https://arxiv.org/abs/1905.12265
11. Laboratories, U.: UL 4600: Standard for Safety for the Evaluation of Autonomous Products. UL Standards (2020), standard for autonomous vehicle safety and validation
12. Liang, M., Yang, B., Hu, R., Chen, Y., Liao, R., Feng, S., Urtasun, R.: Learning lane graph representations for motion forecasting. In: Computer Vision – ECCV 2020. pp. 541–556. Lecture Notes in Computer Science, Springer (2020)
13. Menzel, T., Bagschik, G., Maurer, M.: Scenarios for development, test and validation of automated vehicles. CoRR **abs/1801.08598** (2018), http://arxiv.org/abs/1801.08598
14. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space (2013), https://arxiv.org/abs/1301.3781
15. Mo, X., Ge, J., Wang, Z., Lv, C., Johansson, K.H.: Sparscene: Efficient traffic scene representation via sparse graph learning for large-scale trajectory generation (2025), https://arxiv.org/abs/2512.21133
16. Mo, X., Lou, B., Mao, Z., Huang, Q., Sun, W., Wang, Y., Lv, C.: Traffic scene representation and encoding with graph structure learning and exploration. IEEE Transactions on Intelligent Transportation Systems **26**(10), 16840–16853 (2025). https://doi.org/10.1109/TITS.2025.3582574
17. Mütsch, F., Zipfl, M., Polley, N., Zöllner, J.M.: Exploring semantic clustering and similarity search for heterogeneous traffic scenario graph. In: 2025 IEEE International Automated Vehicle Validation Conference (IAVVC) (2025), https://arxiv.org/abs/2507.05086

18. On-Road Automated Driving (ORAD) Committee: Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles. Standard J3016, SAE International (April 2021). https://doi.org/10.4271/J3016_202104, https://doi.org/10.4271/J3016_202104
19. PEGASUS: Pegasus method. Available online (2019), https://www.pegasusprojekt.de/files/tmpl/Pegasus-Abschlussveranstaltung/PEGASUS-Gesamtmethode.pdf, accessed: June 12, 2026
20. Ries, L., Rigoll, P., Braun, T., Schulik, T., Daube, J., Sax, E.: Trajectory-based clustering of real-world urban driving sequences with multiple traffic objects. In: 2021 IEEE International Intelligent Transportation Systems Conference (ITSC). pp. 1251–1258 (2021). https://doi.org/10.1109/ITSC48978.2021.9564636
21. Roßberg, N., Neumeier, M., Hasirlioglu, S., Bouzouraa, M.E., Botsch, M.: Assessing the completeness of traffic scenario categories for automated highway driving functions via cluster-based analysis (2025), https://arxiv.org/abs/2506.02599
22. for Standardization, I.O.: ISO 21448:2022 – Road Vehicles – Safety of the Intended Functionality (2022), standard document, ISO/TC 22/SC 32
23. Ulbrich, S., Menzel, T., Reschka, A., Schuldt, F., Maurer, M.: Defining and substantiating the terms scene, situation, and scenario for automated driving. In: 2015 IEEE 18th International Conference on Intelligent Transportation Systems. pp. 982–988 (2015). https://doi.org/10.1109/ITSC.2015.164
24. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: International Conference on Learning Representations (ICLR) (2018)
25. Wachenfeld, W., Winner, H.: The Release of Autonomous Vehicles, pp. 425–449. Springer Berlin Heidelberg, Berlin, Heidelberg (2016). https://doi.org/10.1007/978-3-662-48847-8_21, https://doi.org/10.1007/978-3-662-48847-8_21
26. Wilson, B., Qi, W., Agarwal, T., Lambert, J., Singh, J., Khandelwal, S., Pan, B., Kumar, R., Hartnett, A., Pontes, J.K., Ramanan, D., Carr, P., Hays, J.: Argoverse 2: Next generation datasets for self-driving perception and forecasting. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks 2021) (2021)
27. Zhang, Y., Lou, S., Lou, B., Zhang, H., Lv, C.: Adversarial traffic scene generation considering harm, rarity, and ambiguity for autonomous driving testing. Transportation Research Part C: Emerging Technologies **182**, 105426 (2026). https://doi.org/10.1016/j.trc.2025.105426
28. Zipfl, M., Jarosch, M., Zöllner, J.M.: Self supervised clustering of traffic scenes using graph representations (2022), https://arxiv.org/abs/2211.15508
29. Zipfl, M., Jarosch, M., Zöllner, J.M.: Traffic scene similarity: a graph-based contrastive learning approach (2023), https://arxiv.org/abs/2309.09720